

CS683 PA1, Task 2: Matrix Multiplication for Inference

Software prefetching, SIMD, their combination, and a run inside llama.cpp

Name	Roll number
Pranav	24B1074
Atharva	24B0996
Divyaansh	24B0981
Sufiyan	24B0935

Autumn 2026

1 Setup and methodology

Machine. Intel Core i7-13700HX (Raptor Lake). Every run was pinned to CPU 0, a P-core, with `taskset`, governor `performance`. L1-D 48 KB 12-way, L2 1.25 MB per P-core, L3 30 MB, 64 B lines. This part has no AVX-512, so the SIMD widths we could study are 128-bit (SSE) and 256-bit (AVX2 with FMA). We toggled the hardware prefetchers through MSR `0x1a4: wrmsr -a 0x1a4 0xf` disables the L2 streamer, the L2 adjacent-line prefetcher, the L1 streamer and the L1 IP prefetcher, and `0x0` turns them back on. We re-enabled them after every experiment.

Build and harness. Pinned flags `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`, unchanged. The harness takes the median of 3 timed repetitions after one warm-up, checks every stage against `matmul_naive` (relative tolerance 10^{-4}), and reports speedup as $t_{\text{naive}}/t_{\text{stage}}$ on the same input. The prefetch parameters (distance `PF_D` in floats, hint `PF_L`, panel width `JC`) are compile-time macros, so a sweep recompiles rather than editing the source. Counters come from `perf stat` on the P-core PMU: `L1-dcache-load-misses`, `l2_rqsts.miss`, `LLC-load-misses`, `instructions`, and the three `sw_prefetch_access.{t0, t1_t2, nta}` events that count issued software prefetches by hint. We measured instruction counts with an isolated single-call driver, because the harness’s naive reference and warm-ups would otherwise swamp the count.

Layout. $C[i][j] = \sum_p A[i][p] B[j][p]$ with A ($M \times K$), B ($N \times K$, stored transposed) and leading dimensions `lda`, `ldb`, `ldc`. This is ggml’s `mul_mat` form. The contraction index is contiguous in both operands, so the inner loop is a unit-stride dot product.

Kernels. Four kernels appear in this report.

`matmul_simd` (Task 2B, graded) is a 1×4 register-tiled AVX2 micro-kernel. One 8-float vector of row A_i is loaded and reused against 4 rows of B with 4 independent `_mm256_fmadd_ps` accumulators; a horizontal sum runs once per output; scalar tails handle $K \bmod 8$ and $N \bmod 4$. There is no cache blocking.

The scalar prefetch-only kernel (Task 2A, ungraded file `matmul_prefetch_scalar.cpp`) is the naive triple loop with one `_mm_prefetch(b+p+D, hint)` added in the inner loop. Its only purpose is

to isolate software prefetching from every other technique, which is how the assignment document defines Task 2A.

`matmul_prefetch` (Task 2A/2C stage, graded) is the 1×4 SIMD micro-kernel with the j loop blocked into panels of $JC=64$ rows of B , so a $64 \times K$ panel stays in L2 and is reused across all M rows of A , plus `_mm_prefetch` on the four B streams `PF.D` floats ahead. Building with `PF.D=0` compiles the prefetches out and gives the pure-tiling baseline for the with/without comparison.

`matmul_optimized` (Task 2C, graded, and the kernel injected into `llama.cpp`) uses a 4×4 register block with 16 `_m256` accumulators. Each A vector feeds 4 FMAs and each B vector feeds 4, which doubles the arithmetic intensity of the 1×4 kernel. It uses $JC=32$ panels (the best of a 16 to 96 sweep), unrolled accumulation, vectorised tails for ragged M , N , K , and optional `_mm_prefetch` under the same macros. A full SSE port of this kernel (`matmul_optimized128.cpp`, ungraded) served for the width study.

All kernels pass the harness on 256^3 and 1024^3 and on the ragged shapes $37 \times 53 \times 71$, $1 \times 129 \times 130$, $255 \times 257 \times 129$, $5 \times 3 \times 2$ and $3 \times 3 \times 3$.

2 Task 2A: software prefetching

2.1 Prefetching in isolation: the scalar kernel

Table 1: Scalar prefetch-only kernel. Speedup as defined in the assignment, $t_{\text{without}}/t_{\text{with}}$, distance 64 floats, hint T0. Data: `task2a_scalar_prefetch.txt`.

$M = N = K$	128	256	512	1024	2048
without SW prefetch (ms)	0.666	6.06	54.5	436.1	3628.9
with SW prefetch (ms)	1.113	12.18	102.2	857.1	7113.1
speedup	0.60×	0.50×	0.53×	0.51×	0.51×

Adding one prefetch instruction to the scalar loop halves throughput at every size. The distance sweep at $N = 1024$ (8, 16, 32, 64, 128, 256 floats) sits at 0.51 to 0.52× throughout, and the hint sweep (T0, T1, T2, NTA) is just as flat at 0.51 to 0.52×. The `sw_prefetch_access` counters say why. The scalar loop issues one prefetch per element, 5.37×10^9 of them for a 1024^3 product, which is 16 redundant prefetches for every 64-byte line. The inner loop is one load, one multiply-add and one prefetch, so the prefetch doubles the dynamic instruction count of a loop that was already issue-bound, and no distance or hint can buy that back. The prefetches do still work at the cache level. L1-D misses fall by 42 to 45% at every size (40.3 M to 22.6 M at 1024, 235.8 M to 140.7 M at 2048), while L2 and LLC misses stay the same. Turning the hardware prefetchers off changes nothing (0.52× with HW off, 0.53× with HW on), because the cost here is the overhead rather than latency. The vector kernels avoid this by issuing one prefetch per 8-float step.

2.2 Prefetching on the blocked SIMD kernel (`matmul_prefetch`)

Table 2: `matmul_prefetch`: speedup over naive with prefetch on (distance 64, T0), and the assignment-defined speedup $t_{\text{without}}/t_{\text{with}}$. Data: `task2a_sweep.txt`, `task2a_swf_vs_size.txt`.

$M = N = K$	128	256	512	1024	1536	2048
speedup over naive (SW pf on)	13.2×	11.7×	14.1×	12.5×	12.6×	12.1×
GFLOP/s (SW pf on)	70.1	65.9	66.0	60.8	61.2	57.8
without SW pf (ms)	0.046	0.420	3.456	28.70	103.4	246.2
with SW pf (ms)	0.067	0.509	3.910	34.64	111.8	280.7
$t_{\text{without}}/t_{\text{with}}$	0.69×	0.83×	0.88×	0.83×	0.93×	0.88×

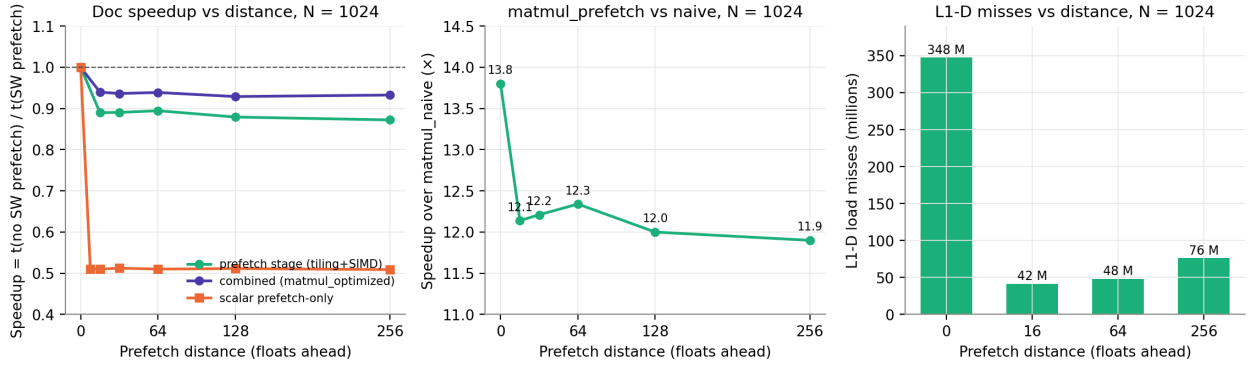


Figure 1: Prefetch distance. Left: assignment-defined speedup versus distance for the three prefetching kernels at $N = 1024$. Middle: `matmul_prefetch` speedup over naive versus distance. Right: its L1-D load misses versus distance (`task2a_cachemiss_params.txt`).

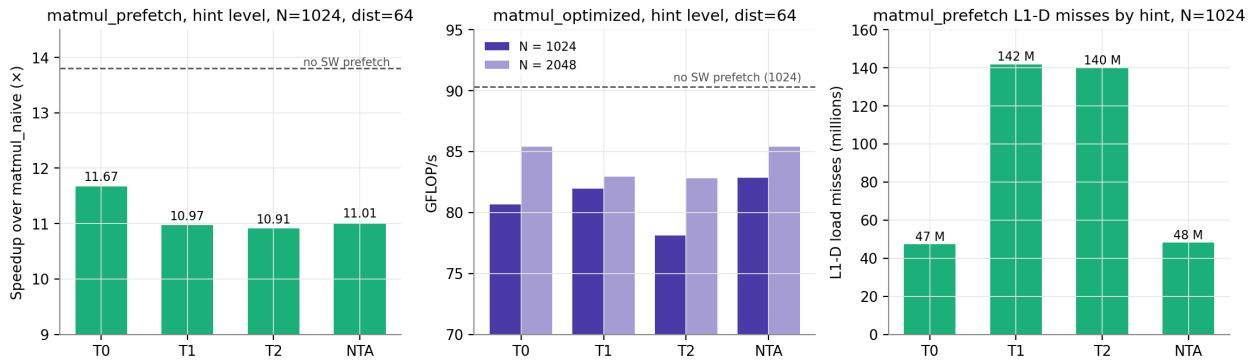


Figure 2: Cache fill level. Left: `matmul_prefetch` at distance 64 for the four hints, dashed line is the no-prefetch build. Middle: the combined kernel at $N = 1024$ and 2048. Right: L1-D misses of `matmul_prefetch` by hint.

Table 3: `matmul_prefetch`, $N = 1024$: cache misses versus prefetch distance and hint (`task2a_cachemiss_params.txt`) and issued software prefetches (`task2a_swpf_counters.txt`).

setting	L1-D misses	L2 misses	LLC misses	sw_prefetch_access (t0 / t1.t2 / nta)
no SW prefetch	347.8 M	418.1 M	23.5 K	11 K / 0 / 0
dist 16, T0	41.6 M	418.1 M	16.2 K	671 M / 0 / 0
dist 64, T0	48.3 M	418.1 M	15.6 K	671 M / 0 / 0
dist 256, T0	76.5 M	418.2 M	32.2 K	671 M / 0 / 0
dist 64, T1	141.8 M	417.9 M	18.0 K	12 K / 671 M / 0
dist 64, T2	139.9 M	418.2 M	18.7 K	12 K / 671 M / 0
dist 64, NTA	48.2 M	420.7 M	18.6 K	12 K / 0 / 671 M

Table 4: `matmul_prefetch`: cache misses versus matrix size, without and with software prefetch (distance 64, T0). Data: `task2a_cachemiss_vs_size.txt`.

N	L1-D misses		L2 misses		LLC misses	
	without	with	without	with	without	with
256	4.75 M	1.04 M	163 K	165 K	3.6 K	3.6 K
1024	348.2 M	48.4 M	418.3 M	418.2 M	25.7 K	16.3 K
2048	2765 M	390 M	3363 M	3385 M	2.35 M	1.07 M

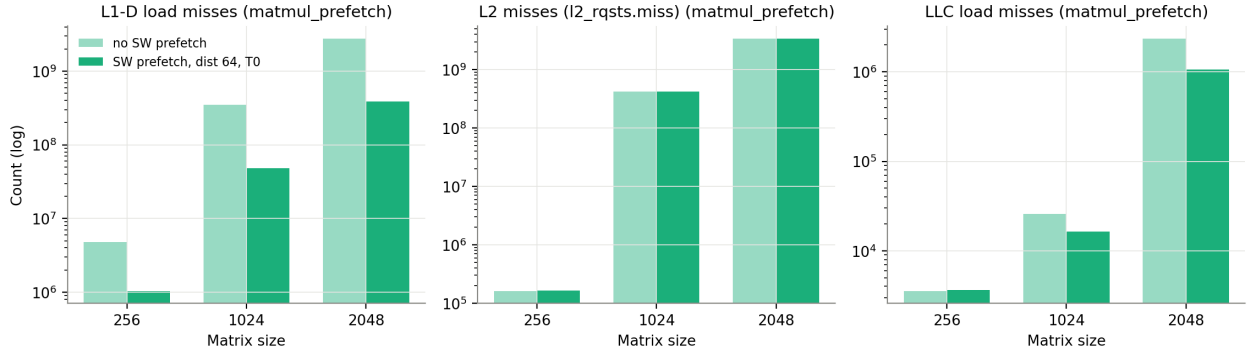


Figure 3: Cache misses at each level versus size for `matmul_prefetch`, with and without software prefetch (log scale).

2.3 Effect of the hardware prefetchers

Table 5: Hardware prefetchers enabled/disabled (MSR 0x1a4), $N = 1024$, software prefetch at distance 64/T0 or compiled out. `task2a_hwpf.txt`, `task2a_scalar_hwpf.txt`, `task2c_hwpf.txt`.

kernel	HW prefetchers	SW prefetch off	SW prefetch on
scalar prefetch-only (speedup vs naive)	off	1.00×	0.52×
	on	1.01×	0.53×
<code>matmul_prefetch</code> (speedup vs naive)	off	13.99×	12.57×
	on	13.37×	12.39×
<code>matmul_optimized</code> (GFLOP/s)	off	73.5	80.6
	on	86.8	82.1

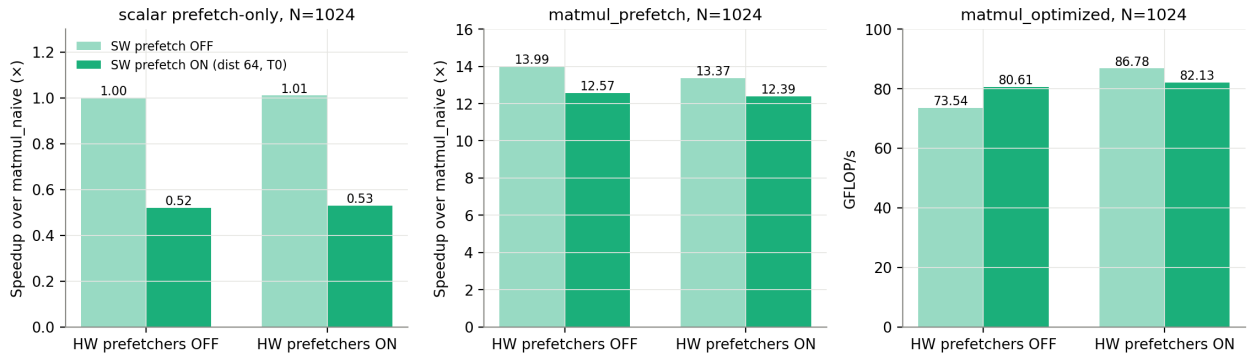


Figure 4: Hardware prefetchers on/off crossed with software prefetch on/off for the three kernels.

2.4 Answers

Q1. Measured the way the assignment defines it ($t_{\text{without}}/t_{\text{with}}$), software prefetching stays below 1.0 at every size for every kernel on this machine: 0.50 to 0.60× for the scalar loop, 0.69 to 0.93× for the blocked SIMD kernel, and 0.94 to 0.98× for the combined kernel (Section 4). The penalty shrinks as the matrix grows, from 0.69 at 128 to 0.88 to 0.93 at 1536 and 2048 for `matmul_prefetch`. At small sizes everything is in L1 or L2 and the prefetch is pure overhead. At 2048 the working set is 48 MB, more than L3, so there is real DRAM latency to hide, and the prefetch hides some of it: LLC misses halve, 2.35 M to 1.07 M (Table 4). Why it never reaches 1.0 is visible in Table 5. With the hardware prefetchers on, the L2 streamer already covers this unit-stride pattern, so software prefetch removes L1 misses that were not on the critical path (348 M to 48 M) while its 671 M extra instructions per 1024^3 product eat issue bandwidth. Against naive, the prefetch stage is a flat 12

to $14\times$ at all sizes, where the un-blocked SIMD kernel decays from $15\times$ to $7\times$. That flatness comes from the cache blocking, not from the prefetch.

Q2. By wall-clock the best distance is 0, i.e. no software prefetch, for all three kernels. Among the non-zero distances the blocked SIMD kernel does best at 64 floats ($12.34\times$, against 12.14 at 16 and 11.90 at 256), and the miss curve has the expected U shape. At 16 floats, one line ahead, L1 misses are lowest (41.6 M) because the line has only just arrived when it is needed. 64 is nearly as good (48.3 M) and gives the core more slack. At 256 floats, 1 KB ahead, prefetched lines get evicted before use, misses climb back to 76.5 M and LLC misses double. We therefore quote 64 floats (4 cache lines) as the best distance when prefetching is used: fastest among the non-zero distances and in the flat bottom of the miss curve. Distance does not change how many prefetches are issued (671 M at every distance), only how far ahead they land.

Q3. T0. On `matmul_prefetch` T0 gives $11.67\times$ against 10.97 (T1), 10.91 (T2) and 11.01 (NTA), and the L1 miss counts show the hints doing what the manual says: T1 and T2 stop the line in L2/L3 and L1 misses triple (47 M to 140 M). NTA matches T0 on L1 misses but is the wrong hint here, because each B panel is reused across all M rows of A and a non-temporal hint invites early eviction. The `sw_prefetch_access` counters confirm that the intended event fires for each hint (Table 3). On the combined kernel the four hints land within 6% of each other (Figure 2). The $JC=32$ panel is already in L2 before the inner loop starts, so the level the hint targets hardly matters; T0 and NTA are best at $N = 2048$ (85.4 GFLOP/s).

Hardware prefetchers. Turning them off costs the blocked SIMD kernel only about 4% (13.99 to $13.37\times$ with SW prefetch off), because the panel blocking already keeps the working set in L2. The combined kernel is the interesting case. With the hardware prefetchers off, software prefetch turns from a 5.4% loss into a 9.6% gain (73.5 to 80.6 GFLOP/s). On this access pattern software and hardware prefetch substitute for each other: the hardware streamer handles unit-stride rows for free, and the software version only pays off when the hardware is absent.

3 Task 2B: SIMD

Table 6: Table 2.1: instruction count (isolated single call) and execution time (harness median) by matrix size and SIMD width. 512-bit is not available on this CPU. Data: `task2b_instr_by_width.txt`, `task2b_simd_width.txt`.

	metric	512 ³	1024 ³	2048 ³
No SIMD (naive)	instructions	838,022,006	6,572,916,527	52,076,483,939
	time (ms)	52.45	435.9	3588.0
SIMD 128-bit (SSE4.2)	instructions	166,896,117	1,201,946,863	9,104,092,474
	time (ms)	6.900	79.31	575.5
	speedup vs no SIMD	7.55 $\times$	5.78 $\times$	6.23 $\times$
SIMD 256-bit (AVX2+FMA)	instructions	65,653,310	395,047,199	2,653,294,523
	time (ms)	4.531	58.37	500.0
	speedup vs no SIMD	11.58 $\times$	7.47 $\times$	7.18 $\times$
SIMD 512-bit		N/A	N/A	N/A
instruction reduction	naive / 128	5.02 $\times$	5.47 $\times$	5.72 $\times$
	naive / 256	12.76 $\times$	16.64 $\times$	19.63 $\times$
	128 / 256	2.54 $\times$	3.04 $\times$	3.43 $\times$

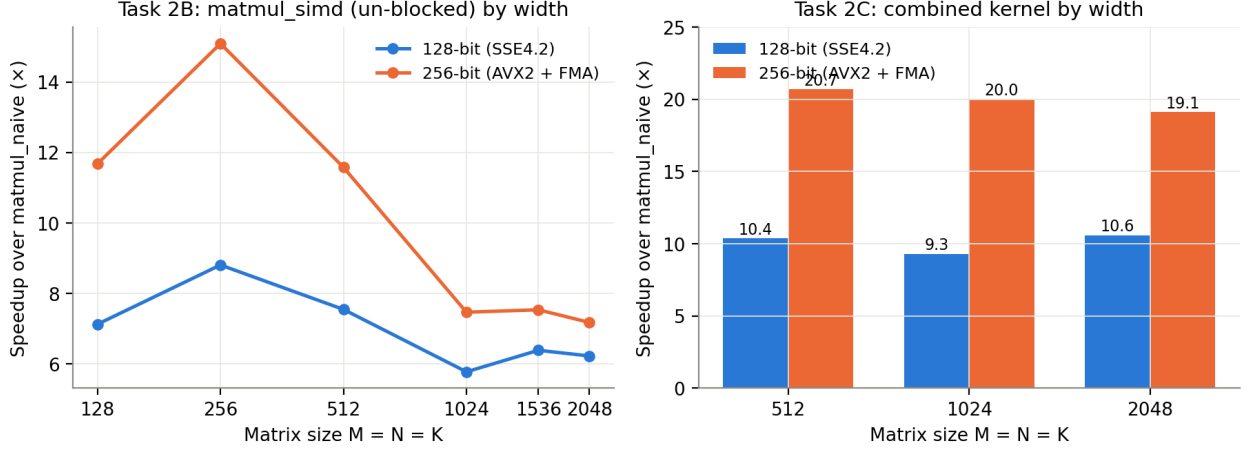


Figure 5: Speedup over naive versus size by SIMD width. Left: the un-blocked `matmul_simd` (Task 2B). Right: the combined kernel with an SSE port (Task 2C).

Table 7: `matmul_simd` (256-bit) speedup over naive across sizes (`task2b_sweep.txt`).

$M = N = K$	128	256	384	512	768	1024	1536	2048
GFLOP/s	79.8	81.1	76.5	58.8	36.0	31.3	34.4	33.0
speedup	14.9×	14.7×	16.7×	11.4×	7.1×	7.0×	7.4×	6.9×

3.1 Answers

Q1. The 256-bit kernel cuts instructions 12.8 to 19.6 $\times$: 8 lanes, times the FMA fusing the separate multiply and add, is about 16 $\times$, and at larger K the per-element loop overhead disappears as well. Time improves only 7 to 17 $\times$, and the speedup has two regimes. Up to 384³, where all three matrices total at most 1.8 MB and sit in L2, the kernel reaches 76 to 81 GFLOP/s and 15 to 17 $\times$. From 512³ on it drops to about 7 $\times$ and 31 to 36 GFLOP/s. The 1×4 kernel streams the whole of B once per row of A with no reuse, so once B is bigger than L2 it is re-read from L3 or DRAM M times and the kernel is bandwidth-bound; at 1024³ L1 misses go from 22 M for naive to 198 M for SIMD. The width comparison says the same thing from the other side. 256-bit beats 128-bit at every size, but the margin shrinks from 1.80 $\times$ at 256 to 1.15 $\times$ at 2048, because a wider vector unit does not help when the loads are the bottleneck. Two caveats on the 128-bit numbers. The SSE4.2 build has no FMA, so it issues separate `_mm_mul_ps` and `_mm_add_ps`, and the 128/256 instruction ratio is 2.5 to 3.4 $\times$ rather than the 2 $\times$ that width alone would give. And the $N = 128$ points take less than a millisecond and are noisy.

Q2. 256-bit (AVX2 with FMA), at every size, with a peak of 16.7 $\times$ at 384³. We could not test 512-bit. On the combined kernel, where cache blocking removes the bandwidth wall, the 256-bit margin is a steady 1.8 to 2.15 $\times$ that does not depend on size (Figure 5 right, Section 4), which is what lane count predicts once the kernel is compute-bound.

Intrinsics. `_mm256_loadu_ps`, because rows of `llama.cpp` tensors are not guaranteed to be 32 B-aligned. `_mm256_fmadd_ps`, one instruction and one rounding, which needs `-mfma`. `_mm256_setzero_ps` for the accumulators. A horizontal-sum helper (`castps256_ps128`, `extractf128`, `add_ps`, two `hadd_ps`, `cvtss_f32`) that runs once per output element, outside the K loop, since AVX2 has no single horizontal-add instruction. The 128-bit variant uses the `_mm*` equivalents. We used single-precision `_ps` intrinsics throughout because SGEMM is `float32`; the `_pd` examples in the appendix would halve the lane count.

4 Task 2C: software prefetching + SIMD

4.1 Speedup versus size for every technique

Table 8: Speedup over naive of each stage, all four measured in the same harness run per size so they share a baseline. The 1536 optimized value is the median of a 3-run recheck (`task2c_recheck.txt`); the first-pass 14.86 $\times$ was a scheduling outlier. Data: `task2c_all_stages_vs_size.txt`.

$M = N = K$	128	256	512	1024	1536	2048
SW prefetch only (scalar)	0.60 $\times$	0.50 $\times$	0.53 $\times$	0.51 $\times$	–	0.51 $\times$
SIMD only (<code>matmul_simd</code>)	12.7 $\times$	14.9 $\times$	13.0 $\times$	7.2 $\times$	7.2 $\times$	6.9 $\times$
SW prefetch + tiling + SIMD (<code>matmul_prefetch</code>)	14.4 $\times$	12.2 $\times$	12.6 $\times$	12.6 $\times$	12.4 $\times$	12.0 $\times$
All combined (<code>matmul_optimized</code>)	13.8 $\times$	18.3 $\times$	19.9 $\times$	19.1 $\times$	18.5 $\times$	18.2 $\times$
GFLOP/s	66.7	100.1	98.7	91.9	88.4	84.2

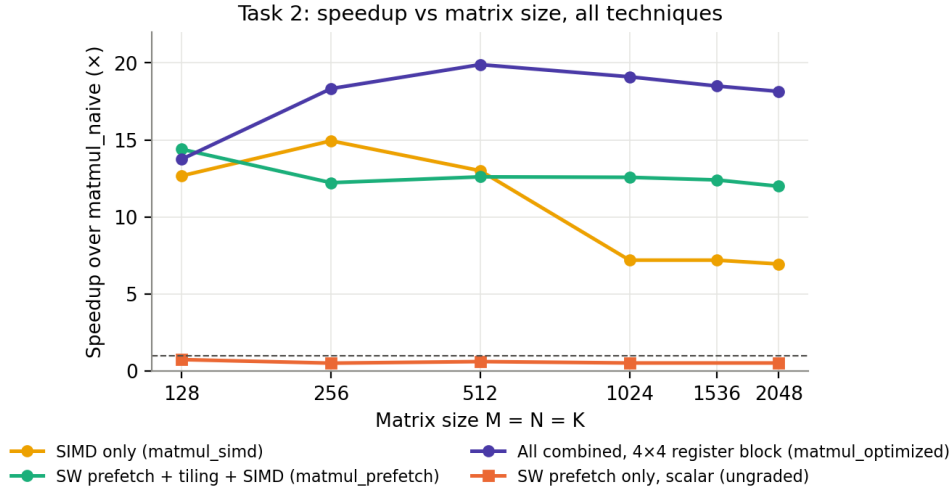


Figure 6: Final comparison: speedup over naive versus matrix size for software prefetching alone, SIMD alone, the prefetching stage built on SIMD and tiling, and the fully combined kernel.

The three graded curves behave differently. SIMD alone falls from 15 $\times$ to 7 $\times$ once B leaves L2. Adding cache blocking and prefetch to the same micro-kernel flattens the curve at about 12 $\times$ but no higher, because the 1×4 kernel does only 4 FMAs per 5 vector loads and cannot use the locality it now has. The combined kernel is flat and high at 18 to 20 $\times$. Its 4×4 register block does 16 FMAs per 8 loads, so the L2-resident panel is consumed at compute speed. At 1024³ the combined kernel (19.1 $\times$) beats the best single stage (12.6 $\times$) and the sum of the individual improvements; neither technique alone gets past about 15 $\times$ at any size. On this Intel machine the harness scores `matmul_optimized` at 18.2 to 19.9 $\times$, which is 55/70 speedup points; the 25 $\times$ top tier was out of reach.

4.2 Repeating the 2A analyses on the combined kernel

Table 9: Combined kernel: prefetch distance and hint sweeps. $N = 1024$ values are medians of 3 runs (`task2c_recheck.txt`); 512 and 2048 from `task2c_width_and_extra_sizes.txt`. GFLOP/s is quoted because the naive baseline drifted between 418 and 470 ms across harness invocations, which perturbs the speedup column more than the kernel itself.

distance (floats)	0	16	32	64	128	256
$N = 512$ GFLOP/s	103.0	101.0	–	100.9	–	96.3
$N = 1024$ GFLOP/s	90.3	84.9	84.6	84.8	83.9	84.2
$N = 2048$ GFLOP/s	89.7	85.1	–	85.1	–	85.7
hint (distance 64)	T0	T1	T2	NTA		
$N = 1024$ GFLOP/s	80.7	81.9	78.1	82.9		
$N = 2048$ GFLOP/s	85.4	82.9	82.8	85.4		

Table 10: Combined kernel: cache misses and throughput without and with software prefetch (distance 64, T0). `task2c_cachemiss.txt`; $N = 2048$ throughput from the 3-run recheck.

N	L1-D misses		L2 misses		LLC misses		GFLOP/s	
	without	with	without	with	without	with	without	with
1024	57.2 M	44.8 M	424 M	424 M	32.6 K	24.5 K	87.0	83.5
2048	635 M	440 M	3408 M	3403 M	1.36 M	0.78 M	82.7	78.2

Speedup versus size and parameters (2A questions 1 to 3 on the combined kernel). The assignment-defined speedup of software prefetch on the combined kernel is $0.98\times$ at 512, $0.94\times$ at 1024 and $0.95\times$ at 2048, a steady 4 to 6% loss that again shrinks a little with size. The best distance is 0 at every size, and the non-zero distances all sit within 1 to 2% of each other; the combined kernel does not care about distance because the panel is already in L2. The best hints are T0 and NTA (85.4 GFLOP/s at 2048), with T1 and T2 about 3% behind. Prefetching does work at the cache level, with 22 to 31% fewer L1 misses and 25 to 43% fewer LLC misses. L2 misses do not change, since prefetch changes when a line arrives and not how many lines the working set needs. But the misses it removes were already hidden behind the 16-FMA inner loop. That is why the graded `matmul_optimized.cpp` ships with `PF_D=0`: the prefetch code is there and tunable, and the measurements said to leave it off.

Hardware prefetchers (2A question 5). Here the combined kernel behaves differently from the earlier ones (Table 5, Figure 4). With hardware prefetching disabled, software prefetch is a 9.6% gain (73.5 to 80.6 GFLOP/s); with it enabled, the same code is a 5.4% loss. The combined kernel is fast enough that memory latency is a visible share of its runtime, so once the hardware streamer is gone the software version has something to do. Software and hardware prefetch substitute for each other on this unit-stride pattern, and the graded default (`PF_D=0`) is the right one for a machine with its hardware prefetchers on, which is the normal case.

Table 11: Instruction count and time, isolated single call (`task2c_instr.txt`).

N	naive instr.	SIMD instr.	combined instr.	naive / combined	SIMD / combined
1024	6,572,879,294	395,350,711	352,332,734	18.7×	1.12×
2048	52,084,965,519	2,656,656,781	2,313,442,978	22.5×	1.15×
wall-clock 1024 (s)	0.466	0.0757	0.0376	12.4×	2.01×
wall-clock 2048 (s)	3.935	0.639	0.245	16.1×	2.61×

Instruction count (2B question 1 on the combined kernel). The combined kernel executes only 12 to 15% fewer instructions than SIMD alone, the saving coming from the 4×4 block’s fewer

loads and horizontal sums, yet it runs 2.0 to 2.6 $\times$ faster. Its advantage is instructions per cycle, not instruction count. Cache blocking lets the same instruction stream retire about twice as fast because the operands come from L2 instead of DRAM. SIMD cuts the count 16 to 20 $\times$; tiling and register blocking raise the rate.

SIMD width (2B question 2 on the combined kernel). With a full SSE port of the 4×4 , JC=32 kernel the comparison is like for like: 10.4 / 9.3 / 10.6 $\times$ for 128-bit against 20.7 / 20.0 / 19.1 $\times$ for 256-bit at 512 / 1024 / 2048, a ratio of 2.0 / 2.15 / 1.8. Unlike the un-blocked kernel, this ratio does not shrink with size, because both widths are now compute-bound. Part of it is FMA fusion, which SSE4.2 lacks, so lane width alone accounts for somewhat less than 2 $\times$. The 128-bit port is correct on the ragged shapes $255 \times 257 \times 129$, $1 \times 129 \times 130$ and $3 \times 3 \times 3$.

4.3 Inside llama.cpp

`make llama-demo` clones `llama.cpp` b5731, builds a stock `llama-cli`, injects `matmul_optimized` into ggml’s single-threaded F32 `mul_mat` path and rebuilds, then runs `stories260K.gguf` (F32, 1.2MB) on both builds with one thread, a fixed seed and the same prompt. Both checks pass:

```
[ok] student matmul_optimized was actually used by ggml
[ok] student output matches stock output token-for-token (kernel is correct)
```

This runs the kernel under ggml’s own tensor strides (non-contiguous `lda`, `ldb`, `ldc`) and on the $M = 1$ generation path, neither of which the microbenchmark covers, so it is the correctness result that matters.

Throughput on this model is at parity within noise, and we do not claim a win or a loss. Over five paired runs the student/native ratio was 0.94 to 1.21 for prompt evaluation (median 1.00) and 0.96 to 1.06 for generation (median 1.04), while the native binary by itself varied 1.44 $\times$ between runs (6.9k to 9.9k tokens/s). The whole inference takes about 6ms, prompt evaluation is timed over five tokens (0.3ms), the model’s matrices fit in L1 so cache blocking has nothing to do, and generation runs at $M = 1$, where the kernel takes its vectorised tail rather than the 4×4 path. Performance claims for the kernel rest on the microbenchmark, where the matrices are big enough for blocking to matter.

5 Summary

Software prefetching (2A) on a scalar loop halves throughput, because a per-element prefetch doubles an issue-bound loop. Issued once per vector on the blocked SIMD kernel it removes 7 $\times$ the L1 misses and half the LLC misses at 2048, with the expected U-shaped miss curve (best 16 to 64 floats ahead) and hint behaviour (T0 best, T1 and T2 triple the L1 misses), yet it still costs 7 to 17% in time because the hardware streamer already covers the pattern. SIMD (2B) cuts instructions 12.8 to 19.6 $\times$ and gives 15 to 17 $\times$ while the data fits in L2, decaying to 7 $\times$ beyond that; 256-bit beats 128-bit at every size. Combining register blocking, SIMD, cache tiling and optional prefetch (2C) gives a flat 18 to 20 $\times$ at 84 to 100 GFLOP/s across all sizes, a like-for-like 2 $\times$ from 256-bit over 128-bit, and twice the IPC of SIMD alone on 12% fewer instructions. Software prefetch only becomes a gain (9.6%) when the hardware prefetchers are disabled, so the shipped kernel keeps it off. Inside `llama.cpp` the same kernel reproduces the native `matmul`’s output token for token.